



数论

数论是NOIP常考内容，他如果单独考就是第一天第一题的难度，但是很多的是放在一道题中，在处理问题的时候要用一些数学知识去处理。

数论的跨度很大，简单的可以到小学奥数题，难的可以到大学高数或者线性代数知识。其中囊括的数学知识很多。



最大公约数

辗转相除法：

$$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$$

求多个数的gcd，等于每一个数和前面几个数的gcd的gcd

因为 $a \% b$ 是 a 除以 b 的余数，所以每一次 a 最多变为以前的一半（至少除以2），所以最坏时间复杂度为 $\log n$ 级别，比如斐波那契数列，一般达不到。

最小公倍数

没有什么直接求解的快速的办法，但是可以借助最大公约数来求解。

$$a*b = \gcd(a, b) * \text{lcm}(a, b)$$

证明：设 $\gcd(a, b) = k$

$a = k*m$ $b = k*n$ 其中 m, n 肯定互质

那么 a, b 的最小公倍数为 $k*m*n$

快速幂

求 $a^n \% p$ 的答案

可以转换成先求 $a^{(n/2)} \% p$ 的答案

$a^{(n/2)} \% p$ 又可以转换成先求 $a^{(n/4)} \% p$

最坏时间复杂度 $\log n$

```
long long quick(int x,int y)
{
    if(y==0)
        return 1;
    long long ans=quick(x,y/2);
    ans=ans*ans%p;
    if(y&1)
        ans=(ans*x)%p;
    return ans;
}
```

快速乘

求n个数之积，答案对p取余，其中 $p \leq 1e18$

两个long long范围内的数相乘会爆掉，但是答案还在long long 范围内，两个数相加是不会超过long long的，所以我们可以把乘法变加法。参考快速幂

```
ll quick(ll a,ll b,ll p)
{
    if(b==0)
        return 0;
    ll ans=quick(a,b/2,p)%p;
    ans=(ans+ans)%p;
    if(b&1)
        ans=(ans+a)%p;
    return ans;
}
```

素数

对于一个数 n 如果他是质数，那么 $2—n-1$ 都不能被他整除，那么如果他不是质数呢？那么 n 就一定可以写成两个数之积，而这两个数都是 n 的因子。

即： $n=p*q$

如果 p 是一个比较小的因子，那么 q 就是一个比较大的因子， p ， q 之间一定有一个数 $\leq \sqrt{n}$ 。

我们思考，如果在 $2—\sqrt{n}$ 中不存 n 的因子，那么是否可以证明 n 就是素数？

完全是可以的。我们可以用反证法：

证明：如果 n 有一个因子 p 大于 $\sqrt{n}$ ，那么 n 就能整除 p ，那么 n/p 就一定小于 $\sqrt{n}$ ，但是在 $2—\sqrt{n}$ 中并不存在 n 的因子，所以 n 一定是质数。

埃式筛素数

我们会发现目前我们会的证明素数只有枚举，素数不容易证明是因为他要枚举完他的所有因子才能判断，在数学中，如果我们没有办法直接证明的时候，我们可以去证明他的反面。

既然素数我们不好直接求解，那我们如果求出所有的合数，那么剩下的数就肯定是素数。

那我们要如何才能判断一个数是否是合数呢？

实际上判断是一个很麻烦的过程，我们完全可以根据合数的定义来构造：一个数的倍数(>1)一定是合数。

埃式筛素数

比如2的倍数：2, 6, 8, 10...

3的倍数：6, 9, 12

4的倍数：8, 12, 16...

我们通过一个数的倍数所生成的数一定是合数，并且只要我们一次枚举这个数，那么合数是不会有遗漏的。

所以我们可以先枚举2—n，再枚举倍数。这样就可以把所有的合数标记出来，剩下未标记的就是素数。

比如10以内的素数：

2的倍数：4 6 8 10

3的倍数：6 9

4的倍数：8

5的倍数：10

6, 7, 8, 9, 10的倍数：无

埃式筛素数

```
int t=sqrt(n)
for(int i=2;i<=t;i++)//枚举因子
{
    if(!book[i])//没有被标记说明是质数
    {
        for(int j=i;i*j<=n;j++)//枚举倍数
            book[i*j]=1;//标记合数
    }
    //如果被标记那么说明是合数，不用合数筛选
}
```

素数

我们再仔细观察一下筛选1—30之间的数的方法，看看是否有数被重复筛选，实际上是有的。

其中12, 18, 24被筛选了2次，30被筛选了3次。理论上来说这种筛选是不应该存在重复筛选的数的，那么为什么会出现呢？我们把这些数分解一下：

$$12=2*2*3$$

$$18=2*3*3$$

$$24=2*2*2*3$$

$$30=2*3*5$$

原来原因在于这些数的因子有很多个，而每次拆分都满足我们筛素数的条件。比如30，用2筛选时： $2*15$, $15>2$ ，用3拆分时。 $3*10$, $10>3$. 用5的倍数拆分时， $5*6$, $6>5$

素数

也就是说这种拆分方法虽然高效，但是仍然还是存在重复个数，而且重复被筛选的次数还不少。

我们假设一个数可以写成 $2^a * 3^b * 5^c \dots$ ，那么他就可能被筛选他的质因子的个数次。着也就导致了这种筛选方法的时间复杂度趋近于 $O(n)$ 但是却高于 $O(n)$ 。

那么我们有没有办法让我们的筛选更加完美呢？也就是说保证我们的每一个数都只会被筛选一次。

欧拉筛素数

一个合数，他可能由多个质因子组成，这是没有办法避免的，但是当我们把一个合数拆分成多个质数相乘的形式，肯定有一个质数是最小的，那么对于任意的一个数，我们能不能只用他的最小质因子来筛选这个合数呢？

比如 $30=2*3*5$ ，他的最小质因子为2，那么我们只在 $2*15$ 的时候把30筛选了，后面 $3*10$ ， $5*6$ 就不再筛选了。

比如在用3做筛选的时候，

$3*3=9$ (筛选)

$3*4=12$ (但是12的最小质因子是2，不筛选)

$3*5=15$ (筛选)

$3*6=18$ (18的最小质因子是2，不筛选)

...

欧拉筛素数

```
for(int i=2;i<=n;i++)//枚举倍数
{
    if(book[i]==0)//如果倍数是素数
    {
        num[total]=i;
        total++;
    }
    for(int j=1;j<=total&& i*num[j]<=n;j++)//枚举找到的质数
    {
        book[i*num[j]]=1;
        if(i%num[j]==0)//说明i的最小质因子是num[j], 后面的不用筛了
            break;
    }
}
```

虽然这两种筛选素数的方法的代码是比较类似，但是他的双重循环的含义完全不一样，两者是反过来的，一定要理解清楚，不能死记硬背，否则后面肯定会混淆。

唯一分解

我们都知道，任何一个合数都可以写成多个质数的乘积，并且这些质数是固定的。也就是唯一分解定理：

唯一分解定理：每一个大于1的正整数均可分解为有限个素数的积，如果不计素因数在乘积中的次序，则分解方式是唯一的。将n的素因数分解中相同的素因子收集到一起，可只每个大于1的正整数n可唯一地写成 $n = p_1^{a_1} * p_2^{a_2} * p_3^{a_3} \dots p_k^{a_k}$ ，其中， $p_1, p_2, p_3, \dots, p_k$ ，是互不相同的素数，而 $a_1, a_2, a_3, \dots, a_k$ 是正整数，上面的分解式称为n的标准分解。

唯一分解

唯一分解定理的正确性也很容易证明，假设一个数的分解方式不止一种，那么 $p=a_1*a_2*\cdots*a_i=b_1*b_2*\cdots*b_i$. 那么等式两边同时除以 a_1 (或者任何一个质因子)，那么左边仍然是一个整数，但是右边，因为所有因子都是质数，那么右边的每一个数都和 a_1 互质(除非相同)，那么等式右边除以 a_1 之后，就不再是一个整数了，所以该分解方式是错误的。

唯一分解

那么如何把一个很大的数唯一分解呢？ $n \leq 10^{12}$

因为一个合数肯定可以继续分解，所以唯一分解最终的答案肯定是质数，所以我们只需要依次枚举这些质数就可以了。但是 10^{12} 以内的质数我们是枚举不完的。

同样，我们考虑一个数如果可以写成两个数之积，那么一定有一个数小于 $\sqrt{n}$ ，一个数大于 $\sqrt{n}$ 。如果我们算出小于 $\sqrt{n}$ 那个数，大的那个数可以用除法得到。因为我们从小到大枚举枚举完了每一个质因子，所以这样最后剩下的大的那个数一定是质数。

唯一分解

```
void fenjie(int x)
{
    int k=2;
    int total=0;
    while(k*k<=x)
    {
        if(x%k==0)
        {
            p[++total]=k;
            while(x%k==0)//对因子重数的其他处理
            {
                sum[total]++;
                x/=k;
            }
        }
        k++;
    }
    if(x>1) p[++total]=x;sum[total]++;
    //再对分解最后的那个质数进行处理
}
```



欧拉函数

求 n 以内与 n 互质的数的个数



欧拉函数

我们还是很容易想到一种很朴素的做法，枚举1— $n-1$ ，判断该数与 n 的最大公约数是否是1，判断最大公约数是否可以用欧几里得算法，也就是辗转相除法。那么我们是否可以只判断到 $\text{sqrt}(n)$ 呢，实际上并不行，因为这里只是问是否有相同因子，而不是一个数是否是另一个数的因子，所以我们必须要枚举完，总的时间复杂度为 $O(n) * \log(n)$ 。

那么，如果 n 特别大呢？ $n \leq 10^{12}$

欧拉函数

我们思考，如果 n 是一个质数，那么 n 以内与 n 互质的数那么肯定就是 $(n-1)$ 个。

那么如果 $n=p*q$ (p, q 都是质数)，那么 n 以内与 n 不互质的数有多少个呢？很明显只有 $p, 2p, 3p \cdots (n/q)p, q, 2q, (n/p-1)*q$ ，因为 p, q 是互质的，所以里面的数都不重复，所以这样的数有 $p*q-p-q+1=(p-1)*(q-1)$ 个。

如果 $n=p*q$ (p, q 不一定是质数，但是 p, q 互质)，在 $1 \cdots (p*q-1)$ 中，只有既与 p 互质 ($f(p)$) 的数，又与 q 互质 ($f(q)$) 的数，才会与 $p*q$ 互质，这样的数一共有 $f(p)*f(q)$ 个。即 $f(p*q)=f(p)*f(q)$ (p, q 互质)

欧拉函数

那么如果一个数 $n = a^p$ (a 是质数), 那么 n 以内与 n 互质的数有多少个?

因为 a 是质数, 那么 n 的质因子只有 a , 那么只有 a 的倍数才和 n 不互质, 那么在 $[1, a]$ 中与 a 互质的数有 $(a-1)$ 个, $[a+1, 2a]$ 中与 a 互质的数有 $(a-1)$ 个, $[a^p - a + 1, a^p]$ 中与 a 互质的数有 $(a-1)$ 个, 那么总的与 a^p 互质的数一共有 $a^p * (1 - 1/a)$ 个。也就是 $a^p - a^{p-1}$ 。

那么对于任意一个数都可以写成 $a_1^{b_1} * a_2^{b_2} * a_3^{b_3} \dots$ 那么每个因子的阶乘都是互质的, 所以可以用积性函数的乘法性质得到

$$f(x) = x * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i);$$

也就是欧拉函数

欧拉函数线性筛

前面我们学习的欧拉函数的相关知识，其中都要用到求 n 以内与 n 互质的数的个数，对于一个数 n ，求解其欧拉函数的时间复杂度为 $O(\sqrt{n})$ ，但是如果我们需要求解 $1--n$ 之间所有数的欧拉函数呢？如果在一个一个去计算太麻烦了，有没有什么更快的办法呢？

可以直接用公式 $f(a*b)=f(a)*f(b)$ (如果 a, b 互质)

如果 n 为质数, $f(n)=n-1$

如果 $n\%i==0$ (i 为质数) $f(n*i)=f(n)*i$

如果 $n\%i!=0$ (i 为质数) $f(n*i)=f(n)*(i-1)$

欧拉函数线性筛

证明：已知 $f(x) = x * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i)$

如果 $x \% i == 0$, 那么 i 就是 x 的质因子, $x * i$ 的质因子和 x 的质因子是相同的, 那么对于 $x * i$

$$f(x * i) = (x * i) * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i)$$

$$\text{即 } f(x * i) = f(x) * i$$

如果 $x \% i \neq 0$

那么 $x * i$ 比 x 多一个质因子 i

$$\text{所以 } f(x * i) = (x * i) * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i) * (1 - 1/i)$$

$$f(x * i) = f(x) * i * (1 - 1/i) = f(x) * (i - 1)$$

欧拉函数

```
for(int i=2;i<=n;i++)
{
    if(!book[i])
    {
        prime[++cnt]=i;
        phi[i]=i-1; // 质数的欧拉函数
    }
    for(int j=1;j<=cnt&& i*prime[j]<=n;j++)
    {
        book[i*prime[j]]=1;
        if(i%prime[j]==0) // 如果可以整除
        {
            phi[i*prime[j]]=phi[i]*prime[j];
            break;
        }
        phi[i*prime[j]]=phi[i]*(prime[j]-1); // 如果不可以整除
    }
}
```



素数的性质

前面我们学习了如何筛选素数以及如何分解素数。素数实际上还有很多重要的性质，也有很多重要的公式和素数有关。



威尔逊定理

如果 p 是一个素数，那么 $((p-1)!+1)\%p=0$

反过来，证明仍然成立。也就是说这是一个充分必要定理。

威尔逊定理

充分性证明(正着证): 略

必要性证明(反着证):

如果 p 不是素数, 那么 $p=a*b$

如果 $a \neq b$

那么 $(p-1)! \% a = 0$ $(p-1)! \% b == 0$

则: $(p-1)! \% (a*b) = 0$

$(p-1)! \% p = 0$ 与 $((p-1)! + 1) \% p = 0$ 矛盾

如果 $a == b$

同理。所以 p 肯定是素数



费马小定理

如果 p 是一个质数，则 $a^{(p-1)\%p}=1$ (a 与 p 互质)
正着一定成立，反着不一定成立



费马小定理

证明：

$1a, 2a, 3a \cdots (p-1)a$ 分别模 p 的余数是 $[1, p-1]$ 的某种排列，也就是说余数都不相同。

这个很好证明，如果 $s*a \% p = t*a \% p$

那么 $(s-t)*a \% p = 0$ ，但是 p 的质数，那么所以小于 p 的数都与 p 互质，所以 $(s-t)$ 与 p 互质， a 与 p 互质，所以证明不成立。

那么我们把这这些数都乘起来

$$a * 2a * 3a * \cdots (p-1)a = (1 * 2 * 3 \cdots (p-1)) \% p$$

$$a^{(p-1)} * (p-1)! = (p-1)! \% p$$

$(p-1)!$ 肯定与 p 互质，所以可约去，所以

$$a^{(p-1)} = 1 \% p \quad \text{即：} a^{(p-1)} \% p = 1$$

费马小定理

费马小定理的逆定理不一定成立。

如果满足 $a^{(p-1)} \% p = 1$, 则 p 一定是素数。

比如 341, 561, 1005..., 这样的数被称为伪素数, 而且伪素数的个数为无限多个, 所以不能用该定理直接判断一个数是否是素数。就算不停的更换 a , 也有一些合数永远满足对于所有数 a , 都有 $a^{(p-1)} \% p = 1$

Miller-Rabbin

对于单个数，我们要判断其是否是素数，除了前面讲的试除法，但是效率很低，虽然费马小定律判断很快，但是判断并不准确。这里我们介绍一中准确并且快捷的办法——Miller-Rabbin测试。

首先介绍一个二次探测定理：

若 p 为素数， $a^2 \% p = 1$ ，则 $a \% p = 1$ 或者 -1

证明： $(a^2 - 1) \% p = 0$

$(a+1) * (a-1) \% p = 0$

$a+1=0$ 或者 $a-1=0$ ，所以 $a=-1$ 或者 $a=1$

Miller-Rabbin

对于质数 $p(p>2)$ ，他一定是奇数，那么 $p-1$ 一定是偶数，那么一定可以写成 2^k*t 的形式。

$$a^{(p-1)} \% p = 1 \rightarrow (a^{(p-1)} - 1) \% p$$

$$(a^{((p-1)/2}-1) * (a^{((p-1)/2}+1)) \% p = 0$$

所以该算法流程为：

选择一个或多个 a ，给定奇数 p ，先判断 $a^{(p-1)} \% p$ 是否等于1，如果不等于，那么就一定是合数，否则继续检测。我们先把 p 分解成 2^k*t 的形式，那么下次验证 p 的 $2^{(k-1)}*t \% p$ 答案是否是1或者 $p-1(-1)$ ，然后已知重复验证下去，只要不能分为止。

Miller-Rabbin

每一次费马小定律的检测正确率虽然很低，但是对于伪素数而言，是一定会错的，但是如果我们用费马小定律+二次检测定理组成的miller-rabbin算法来检测，答案的正确率可以达到99.9999%，如果使用4个及以上a的值，我们可认为在long long范围内答案一定是正确的(不包含高精)。

欧拉定理

如果 a, n 互质，那么 $a^{f(n)} \% n = 1$

$f(n)$ 表示欧拉函数，即 n 以内与 n 互质的数的个数



计数原理

计数原理是数学中的重要研究对象之一，包括抽屉原理，加法原理，乘法原理，容斥原理等等。



计数原理

(一) 抽屉原理:

把 $n+1$ 件东西放入 n 个抽屉，则至少有一个抽屉里放了两件或两件以上的东西。

(二) 加法原理

若事件 A_1 有 p_1 种产生方式，事件 A_2 有 p_2 中产生方式...事件 A_n 有 p_n 种产生方式，则事件 A_1 或 A_2 ...或 A_n 有 $p_1+p_2+\dots+p_n$ 种产生方式。

比如从A地到B地，有 x 趟火车，有 y 趟汽车，问从A地到B地有多少种交通方式： $x+y$

一定要注意事件A产生的方式和事件B产生的方式不能有重叠，否则计算是错误的。

计数原理

(三) 乘法原理:

如果事件A有 p 种产生方式，事件B有 q 种产生方式，
则事件A与B有 $p*q$ 种产生方式

使用乘法原理的条件是事件A与事件B相互独立，即
他们之间产生的方式彼此无关。

比如A地到B地 x 种交通方式，B地到C地有 y 种交通方式，
那么从A地经过B地到达C地有多少种不同的到达方式。
答案是 $x*y$ 种

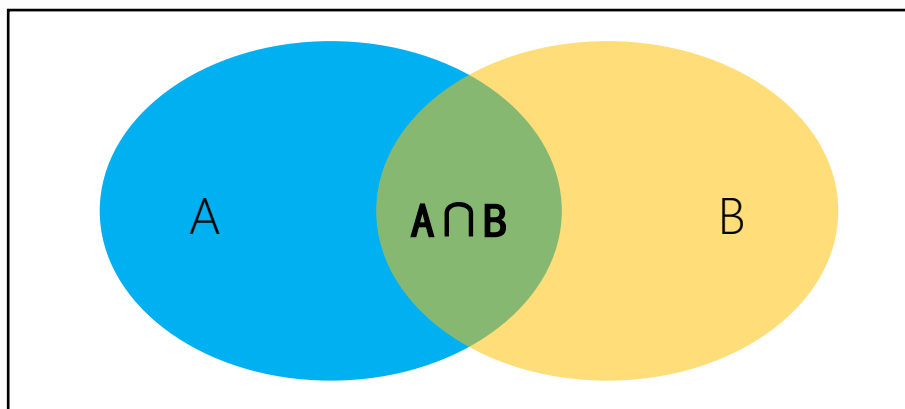
计数原理

(三) 容斥原理:

如果被计数的事物有A,B两类,那么,A类和B类元素总和=属于A类元素的个数+属于B类元素个数-既是A类又是B类元素个数。即为:

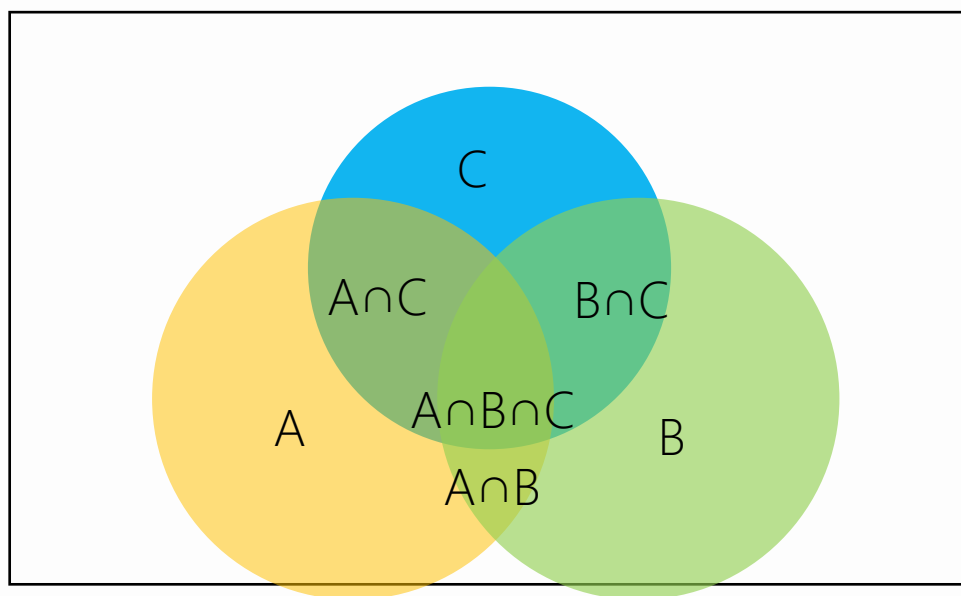
$$A \cup B = A + B - A \cap B$$

班上有x人喜欢画画,有y人喜欢运动,有z人即喜欢画画又喜欢运动,那么喜欢画画和喜欢运动的人一共有多少人: $x+y-z$



计数原理

如果是三个事件呢，两两事件有相互重叠部分，三个事件也有共同的部分。



我们一点一点去拆分就会得到结果：

$$A \cup B \cup C = A + B + C - A \cap B - A \cap C - B \cap C + A \cap B \cap C$$



容斥原理

所以容斥原理的规律为奇加偶减，如果重叠了奇数次，那么系数前面就是“+”，如果是偶数，那么系数前面就是“-”。



排列和组合

在组合数学的研究对象中，根据有无顺序，一般分为排列问题和组合问题。排列与组合的根本区别在于前者与元素的顺序有关，后者与元素的顺序无关。

在排列与组合的问题中，经常会出现计数问题，解决计数问题的思路一般有以下三种：

(1) 只取要的。把各种符合条件的情形枚举出来，利用加法原理求和

(2) 先全部取，再减去不要的。用所有可能情况减去不符合条件的情况

(3) 先取后排。先把各步中符合条件的组合或排列计算出来，再根据乘法原理求积

选排列

从 n 个互不相同的元素的集合 S 中，有序的选择 r 个元素($r \leq n$)，叫做 S 的一个 r 排列。记做 $A(n, r)$

利用乘法原理我们可以知道，对于第一个元素，有 n 种选择，对于第二个元素，因为前面已经选了一个元素了，所以只剩下 $n-1$ 种元素可以选择...同理，对于选择的第 r 个元素，还剩下 $(n-r+1)$ 个元素可以选择。

并且，每一个位置的元素的元素都是相互独立的，所以 $A(n, r) = n * (n-1) * (n-2) * \dots * (n-r+1)$

也可以写成 $n! / (n-r)!$

选排列

如果这 n 个元素中，有 n_1 个元素相同， n_2 个元素相同 $\cdots n_i$ 个元素相同，并且 $n_1+n_2+\cdots+n_i=n$ ，那么从这 n 个元素中选择 r 个元素的排列方式有多少种？

选排列

首先，如果不考虑重复问题，那么从 n 个元素中选择 r 个元素的排列方式为 $A(n, r)$ ，但是因为存在重复元素。比如从2 2选择2个数的排列方式 $A(2, 2)=2$ ，但是实际上只有一种，既然是排列，那么肯定会排列第一个2在前面和第二个2在前面这两种情况，但是实际上这两种情况只能算一种，对于有 x 个相同元素，这样重复的次数为 $x!$ 种，即重复次数的全排列，所以应该除去。所以最终答案应该是

$$A(n, r) / (n_1! * n_2! * \dots * n_i!)$$



选排列

有3个相同的黄球，2个相同的蓝球，4个相同的白球排成一排，问：有多少种不同的排法。

1260





选排列

将1, 2, 3, 4, 5五个数进行排列，要求4一定要排在2的前面，问：有多少种排法。



选排列

我们先考虑总的排列方法有 $A(5, 5) = 5! = 120$ 种，再去除掉不合理的方法，其中4在2前面和2在4前面的机会是均等的，也就是说全排列中有一半的数是4在2前面，有一半的数是2在4前面，所以应该 $/2 = 60$ 就是最终答案

错排列

对于 $\{1, 2, \dots, n\}$ 的全排列中，如果对于任意的 i ，都有 $a_i \neq i$ ，那么该排列就是 $\{1, 2, \dots, n\}$ 的错排列。记做 D_n 。
那么对于 $1, \dots, n$ ，错排列有多少种呢？

错排列

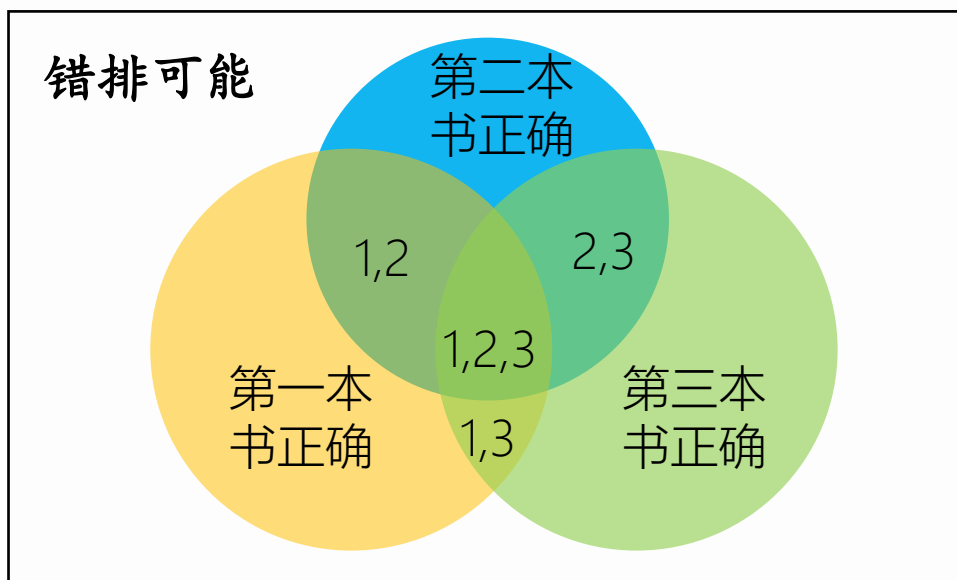
首先，全排列有 $n!$ 种

至少有一本书 a_i 就在第 i 个位置上的情况有 $(n-1)!$ 种

至少有两本书都在正确的位置上有 $(n-2)!$ 种

至少有 k 本书都在正确位置上的情况有 $(n-k)!$ 种

那么我们可以考虑容斥原理，情况为：



所有排列可能

错排列

所以该题就是一个裸的容斥原理：

全排列-一本书正确的排列+其中两本书正确的排列-
其中三本书正确的排列...

即 $A(n, n) = C(n, 1) * (n-1)! + C(n, 2) * (n-2)! -$
 $C(n, 3) * (n-3)! \dots$

我们把这个式子化简就得到 $A(n, n) * (1 - 1/1! + 1/2! -$
 $1/3! + 1/4! - \dots + (-1)^n / n!)$

这就是错排列的公式

错排列

如果你觉得这个公式比较复杂，也可以用递推的形式来计算。设 $f(x)$ 为 x 个数的错排列，对于第一个数，至少不是1都可以，那么就有 $(n-1)$ 种排列方式，那么对于剩下的数，如果1在



错排列

书架上有6本书，编号分别为1—6，取出来再放回去，要求每本书都不在原来的位置上，问有多少种排列方法：

265种



圆排列

对于任意的一个排列，如果依次把队首元素移动到队尾，最多移动 n 次之后就会重复，也就是说这在全排列中算 n 次，但是在圆排列中因为这是一个环，所以只算一种，即圆排列公式为 $A(n, r)/r$



圆排列

有男女各5人，其中3对是夫妻，沿着10个位置的圆桌就座，若每对夫妻都要做到相邻的位置，问有多少种做法。



圆排列

我们可以先考虑没有限制的做法，也就是另外4个人和三对夫妻中每队夫妻中的一人，也就是7人可以随意做，圆排列为 $A(7, 7)/7$ ，夫妻中剩下的一人可以随意选择另一方的一边做，都有两种，所以答案为 $A(7, 7)/7 * 2 * 2 * 2 = 5760$ 种

组合

从 n 个元素的集合 S 中，无序的选取出 r 个元素，叫做 S 的一个 r 组合，记做 $C(n, r)$ 。

组合是无序的，排列是有序的，选择出来的 r 个元素可以随意排列，在排列中算 $r!$ 种，但是在组合中只算一种，所以我们就得到了组合数的公式
$$C(n, r) = A(n, r) / r! = n! / ((n-r)! * r!)$$

组合

通过组合数的公式，我们可以得到以下几个等式：

$$C(n, r) = C(n, n-r)$$

$$C(n, r) = C(n-1, r) + C(n-1, r-1)$$

$$C(n, 0) + C(n, 1) + \cdots + C(n, n) = 2^n$$

组合

前两个等式直接可以用组合数的公式来证明，第三个式子也很容易证明，我们把 $C(n, i)$ 看做 n 位二进制上有 i 个1的方案数，那么 $C(n, 0)$ 表示 n 位二进制数全是0的方案， $C(n, 1)$ 表示 n 位二进制上有一个1的方案， $C(n, n)$ 表示 n 位二进制上有 n 个1的方案，所有方案加起来就是 n 位二进制的所有可能性，即为 2^n



组合

一班有10名同学，二班有8名同学，要在每个班级选出2名学生参加一个比赛，一共有多少种方案。

1260





组合

某班有10名同学，其中有4名女同学，要在班级选出3名学生参加比赛，其中至少有一名女同学，有多少种选择方案。



组合

该题既可以分情况讨论，即刚好一名女生方案+刚好两名女生方案+刚好三名女生方案

$$C(4, 1) * C(6, 2) + C(4, 2) * C(6, 1) + C(4, 3) * C(6, 0) = 100$$

也可以用减法：总方案—没有女生参加的方案

$$C(10, 3) - C(6, 3) = 100$$

捆绑法

对于某几个元素要求相邻的排列问题，可以先将相邻的元素捆绑在一起，看做一个大的元素，与其他元素排列，然后再对相邻的元素内部进行排列。

7个人站成一排照相，要求甲乙丙三人相邻，问有多少种排列方法：

我们先把甲乙丙三人捆绑到一起看做一个元素，与其余四人共五个元素做全排列，共 $A(5, 5)$ 种排法，再对甲乙丙三人进行全排列 $A(3, 3)$ ，乘起来就是答案。

9个人排成一排，有两个个老师，3个男学生，4个女学生，三个男学生要求相邻，4个女学生也要求相邻，问有多少种排列方法。

插空法

对于某几个元素不相邻的排列问题，可先将其他元素排好，然后再将不相邻的元素在已排好的元素之间及两端的空袭之间插入即可。

7人站成一排照相，要求甲乙丙三人不相邻，分别有多少种站法。

我们可以先让其他4人站好，共 $A(4, 4)$ 种排法，再在这4人之间及两端共5个空位选三个位置让甲乙丙三人插入，肯定不相邻，有 $A(5, 3)$ 种方法，乘起来就是答案。

捆绑法主要用来处理相邻元素排列问题，插空法恰好相反，用来处理不相邻元素的排列问题。

除法

对于某几个元素顺序一定的排列问题，可以先将这几个元素与其他元素一同进行排列，然后用总的排列数除以这几个元素的全排列数

7人排队，甲在乙前面的排法有几种：

先不考虑限制条件，有 $A(7, 7)$ 种可能，而甲乙之间的排法有 $A(2, 2)$ 种，甲在乙前面的排法只有一种复合条件，所以直接用 $A(7, 7) / A(2, 2)$ 就可以了。

直排法

如果若干个元素需要排成几排，问有多少种排法。比如7个人，第一排3人，第二排4人，问有多少种不同的排法。
实际上和排成一排，有七个人是一样的，答案是 $A(7, 7)$ 。

消序

有4名男生，3名女生，高矮互不相等，将他们排成一排，要求从左到右，女生由矮到高排，问有多少种排法。

其实很简单，女生从矮到高只有一种排法，男生随便排就可以了，最终答案 $A(7, 4)$

隔板法

把 n 个相同的小球放入 m 个不同的盒子，不允许盒子为空，但是球必须放完，问有多少种不同的方法。

我们可以用隔板法，相当于用 $m-1$ 个隔板把 n 堆小球分成 n 堆，其中有 $n-1$ 个位置可以选择，所以答案为 $C(n-1, m-1)$

但是隔板法不能用于有盒子为空这种情况，如果可以盒子可以为空，那么很明显不成立。那该怎么办呢？

我们转换一下，加上 m 个小球，保证每个盒子都有一个小球，这样就满足隔板法的要求了(最后每个盒子去掉一个小球就是满足原始条件的答案了)，所以有 $C(n+m-1, m-1)$ 种情况

处理排列组合问题的方法还有很多，自行研究。

住店法

有的时候排列可以重复，一般情况下有两类元素，一类元素不可以重复，一类元素可以重复，我们可以把不能重复的元素看做是客，可以重复的元素看做是店。再用乘法原理求解

七名学生争夺五项冠军，每项冠军只能有一个人获得，那么获得冠军的可能的种数有多少种



练习题

洛谷	P3197	越狱
洛谷	P2290	树的计数
洛谷	P2822	组合数问题
洛谷	P3048	牛的IDCow IDs



越狱

监狱有连续编号为 $1 \cdots N$ 的 N 个房间，每个房间关押一个犯人，有 M 种宗教，每个犯人可能信仰其中一种。如果相邻房间的犯人的宗教相同，就可能发生越狱，求有多少种状态可能发生越狱。

输入：

2 3

输出：

6

6种状态为 (000) (001) (011) (100) (110) (111)

$1 \leq M \leq 10^8$

$1 \leq N \leq 10^{12}$

解题思路

我们会发现直接求越狱的情况很麻烦，因为只要有其中相邻两囚犯越狱那么就算越狱，并且不同的人越狱算不同的情况，所以需要分很多种情况来讨论。但是，我们会发现，不越狱的情况是固定的，即所有人都不冲突。

我们先求出总的可以信仰的宗教的方案： n 个房间，每个房间有 m 中可能，答案就是 m^n

我们再算出不可能越狱的情况，即只要相邻两个人的信仰不一样就可以了，第一个可能情况有 m 种，第二个人因为不能和第一个人相同，那么只能有 $(m-1)$ 种，第三个人不能和第二个人冲突，所以也是 $(m-1)$ 种，以此类推，答案为 $m(m-1)^{n-1}$

所以最终答案为 $m^n - m(m-1)^{n-1}$

组合数问题

组合数 C_n^m 表示的是从 n 个物品中选出 m 个物品的方案数。举个例子，从 $(1, 2, 3)$ 三个物品中选择两个物品可以有 $(1, 2), (1, 3), (2, 3)$ 这三种选择方法。根据组合数的定义，我们可以给出计算组合数 C_n^m 的一般公式：

$$C_n^m = \frac{n!}{m!(n-m)!}$$

其中 $n! = 1 \times 2 \times \cdots \times n$ ；特别地，定义 $0! = 1$ 。

小葱想知道如果给定 n, m 和 k ，对于所有的 $0 \leq i \leq n, 0 \leq j \leq \min(i, m)$ 有多少对 (i, j) 满足 C_i^j 是 k 的倍数。

虽然题目中“好心”的给了你一个计算组合数的公式，但是如果使用这个公式来计算，最多只有30分。

$C(n, m)$ ：从 n 个数里面选取 m 个数，有多少组合方式

$A(n, m)$ ：从 n 个数中选取 m 个数进行排序，有多少种排列方法。

杨辉三角形

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
.....

$$C(5,0)=1$$

$$C(5,1)=5$$

$$C(5,2)=10$$

$$C(5,3)=10$$

$$C(5,4)=5$$

$$C(5,5)=1$$

所以实际上 $C(n, m)$ 就是杨辉三角形中第 $(n+1)$ 行第 $(m+1)$ 个数。

$$C(n, m) = C(n-1, m-1) + C(n-1, m);$$

组合数公式

按照前面的办法，预处理出n以内所有的组合数时间复杂度为 $O(n^2)$ ，查询为 $O(1)$ ，空间复杂度为 $O(n^2)$ ，如果n特别大，查询次数比较小的时候就会爆空间，爆时间。

$$C_n^m = \frac{n!}{m!(n-m)!} \quad \longrightarrow \quad C(n,m) = (n-m+1)/m * C(n,m-1)$$

但是这样做有一个更加严重的问题，组合数他的增长实际上是很快的，很多时候我们需要的答案一般是对一个数取余的结果，但是出发取余不满足分配率。即

$$(a/b) \% p \neq a \% p / b \% p$$

所以需要用到一个新的知识。



组合数求解

组合数因为涉及到阶乘，所以数据稍微大一点就超过 `long long` 了，但是最后的答案一般会对一个大质数取余。根据不同的数据范围我们可以选择不同的求解方法。



计算系数

题目描述

给定一个多项式 $(by+ax)^k$ ，请求出多项式展开后 $x^n y^m$ 项的系数。

输入输出格式

输入格式：

输入文件名为factor.in。

共一行，包含5 个整数，分别为 a , b , k , n , m ，每两个整数之间用一个空格隔开。

输出格式：

输出共1 行，包含一个整数，表示所求的系数，这个系数可能很大，输出对10007 取模后的结果。

二项式定理

$$(x+y)^0=1$$

$$(x+y)^1=1x+1y;$$

$$(x+y)^2=1x^2+2xy+1y^2;$$

$$(x+y)^3=1x^3+3x^2y+3xy^2+1y^3;$$

$$(x+y)^4=1x^4+4x^3y+6x^2y^2+4xy^3+1y^4;$$

$$(x+y)^5=1x^5+5x^4y+10x^3y^2+10x^2y^3+5xy^4+1y^5;$$

$$(a+b)^n = C_n^0 a^n + C_n^1 a^{n-1} b + \dots + C_n^r a^{n-r} b^r + \dots + C_n^n b^n$$



练习题

NOIP 2016 组合数问题

NOIP 2011 计算系数



组合数求解

如果要求我们求 $C(n, m)$ ，并且 n, m 都是 10^w 级别的，前面的递推的做法肯定会超过，所以我们需要快速求解。

$$C_n^m = \frac{n!}{m!(n-m)!}$$

$$C(n, m) = (n-m+1) / m * C(n, m-1)$$

根据上面的公式我们如果知道第 n 行第 i 个值，那么我们是可以直接算出来第 n 行第 $i+1$ 个值的，但是这里涉及到除法，我们不能直接取模。所以我们需要找到 m 关于取模的数 p 的逆元。

逆元

什么是逆元呢？简单来说就是：

如果 $(a/b) \% p$ 始终等于 $(a * c) \% p$ ，那么就称之为 b 和 c 相对于 p 互为逆元。逆元是相对的。

正式解释：

如果 c 为 b 的逆元，那么 $bc \equiv 1 \pmod{p}$

即 $bc \% p = 1$

所以 $(a/b) \% p = (a/b * 1) \% p = (a/b * (bc)) \% p = (ac) \% p$

所以 you 只需要知道，如果你需要求很多个 a/b 之类的数相乘对 p 取余的答案的时候，转换成 $*b$ 的逆元就可以了。这样你就可以边乘边取余了。

逆元

既然我们知道了逆元的神奇的作用，关键是我们怎么知道 a 相对于 p 的逆元是多少呢？

实际上就可以用我们今天这节课学习的知识得到答案，目前来说，我们暂时只能求 p 为大质数的情况下的逆元，即答案始终和 p 互质的情况下的逆元。

(1) 费马小定律

如果 p 是质数，那么 $a^{(p-1)} \% p = 1$

逆元则是 $a * b \% p = 1$

所以 a 相对于 p 的逆元就是 $a^{(p-2)}$

再用快速幂求一遍逆元就可以了

(2) 拓展欧几里得原理

$$ax + by = \gcd(a, b)$$

如果 b 为大质数，则 $\gcd(a, b) = 1$

相当于 $ax \% b = 1$ ，只需要求出 x 的最小正整数解就可以了。

一般情况下，考试的时候 p 都是一个很大的质数，所以第二种方法高一第一种，但是第一种方法书写简单，容易理解。

线性筛逆元

费马小定律只能用于 p 为素数的时候，拓展欧几里得只能用于两个数互素的时候，如果是其他情况，可以考虑线性筛逆元。不过线性筛逆元主要用于求区间 $[1, n]$ 的数相对于 p 的逆元。

公式：

$$\text{inv}[1]=1; \quad \text{inv}[i]=(p-p/i)*\text{inv}[p\%i]\%p;$$

然后设 $p = k * i + r, (1 < r < i < p)$ 也就是 k 是 p/i 的商， r 是余数。

再将这个式子放到 $(\text{mod } p)$ 意义下就会得到：

$$k * i + r \equiv 0 \pmod{p}$$

然后乘上 i^{-1}, r^{-1} 就可以得到：

$$k * r^{-1} + i^{-1} \equiv 0 \pmod{p}$$

$$i^{-1} \equiv -k * r^{-1} \pmod{p}$$

$$i^{-1} \equiv -\left\lfloor \frac{p}{i} \right\rfloor * (p \bmod i)^{-1} \pmod{p}$$

于是，我们就可以从前面推出当前的逆元了。

拓展欧几里得原理

求 $ax+by=\gcd(a, b)$ 的整数解

$$ax+by=\gcd(a, b)$$

$$bx'+(a\%b)y'=\gcd(b, a\%b)$$



$$ax+by=bx'+(a-a/b*b)y'$$

要对于所有的 (a, b) 都成立, 所以

$$ax+by=ay'+b(x'-a/b*y')$$

最终得到:

$$x=y'$$

$$y=x'-a/b*y'$$

即只要我们知道其中一组解,
就可以推出其他所有的整数解

对于

$$bx+(a\%b)y=\gcd(b, a\%b)$$

循环下去肯定有 $a\%b=0$

$$\text{此时 } bx=\gcd(b, 0)=b$$

所以可以得到一组解

$$x=1, y=0;$$

同余方程

题目描述

求关于 x 的同余方程 $ax \equiv 1 \pmod{b}$ 的最小正整数解。

输入输出格式

输入格式：

输入只有一行，包含两个正整数 a, b ，用一个空格隔开。

输出格式：

输出只有一行，包含一个正整数 x_0 ，即最小正整数解。输入数据保证一定有解。

输入输出样例

输入样例#1： [复制](#)

```
3 10
```

输出样例#1： [复制](#)

```
7
```

同余方程

该题需要变形，拓展欧几里得原理是求 $ax+by$ 得一组整数解，但是本题只有一个未知数 x ，但是实际上我们可以把 $ax \equiv 1 \pmod{b}$ 看做 $ax-by=1$ 对于 x 的最小正整数解。
 exgcd 中是 $ax+by$ ，没关系我们可以看做 $ax+by'$ ($y'=-y$)，最终求出 (x, y) 的值后，把 $y*(-1)$ 即可

```
void exgcd(int a,int b,int &x,int &y)
{
    if(b==0)
    {
        x=1;
        y=0;
        return ;
    }
    exgcd(b,a%b,y,x);
    y=y-(a/b)*x;
    return ;
}
```

同余方程

```
void exgcd(int a, int b)
{
    if(b==0)
    {
        x=1;
        y=0;
        return ;
    }
    exgcd(b, a%b);
    k=x; //k为原来的x
    x=y;
    y=k-a/b*y;
    return;
}
```

仔细观察，为什么这两种写法的y的赋值有区别。

同余方程

当我们求出一组解之后，我们该如何求出剩下的解。

$$ax_1 + by_1 = ax_2 + by_2$$

$$a(x_1 - x_2) = b(y_2 - y_1)$$

$$a/\gcd(a, b) * (x_1 - x_2) = b/\gcd(a, b) * (y_2 - y_1)$$

这里 $a/\gcd(a, b)$ 和 $b/\gcd(a, b)$ 是互质的，要使等式相等，那么 $(x_1 - x_2)$ 是 $b/\gcd(a, b)$ 的倍数， $(y_2 - y_1)$ 是 $a/\gcd(a, b)$ 的倍数，所以解集为
 $(x + b/\gcd(a, b) * k, y - a/\gcd(a, b) * k)$



练习题

POJ1061:青蛙的约会
NOI2002 荒岛野人



组合数求解

如果 n, m 更大，达到 $100w$ 甚至 $1000w$ 级别呢，我们又该如何求解呢？

我们可以直接用原组合数公式 $n! / ((n-m)! * m!)$ ，我们只需要算出任意一个 $n!$ 相对于 p 的逆元就可以了。

那么如何快速求出所有数的阶乘逆元呢？

阶乘逆元

设 $(i+1)!$ 的逆元为 $\text{inv}[i+1]$, 那么

$$\text{inv}[i+1] = 1 / (i+1)!$$

$$\text{inv}[i+1] * (i+1) = 1 / i! = \text{inv}[i]$$

所以阶乘里面的线性递推公式为

$$\text{inv}[i+1] * (i+1) = \text{inv}[i]$$

所以我们可以先用其他方法求出 $n!$ 的逆元, 先求出 $n! \% p$ 的答案, 然后用费马小定律求出其逆元, 就是 $n!$ 的逆元, 然后递推出 $(n-1)! \dots 1!$ 的逆元。

线性筛逆元

费马小定律只能用于 p 为素数的时候，拓展欧几里得只能用于两个数互素的时候，如果是其他情况，可以考虑线性筛逆元。不过线性筛逆元主要用于求区间 $[1, n]$ 的数相对于 p 的逆元。

公式：

$$\text{inv}[1]=1; \quad \text{inv}[i]=(p-p/i)*\text{inv}[p\%i]\%p;$$

然后设 $p = k * i + r, (1 < r < i < p)$ 也就是 k 是 p/i 的商， r 是余数。

再将这个式子放到 $(\text{mod } p)$ 意义下就会得到：

$$k * i + r \equiv 0 \pmod{p}$$

然后乘上 i^{-1}, r^{-1} 就可以得到：

$$k * r^{-1} + i^{-1} \equiv 0 \pmod{p}$$

$$i^{-1} \equiv -k * r^{-1} \pmod{p}$$

$$i^{-1} \equiv -\left\lfloor \frac{p}{i} \right\rfloor * (p \bmod i)^{-1} \pmod{p}$$

于是，我们就可以从前面推出当前的逆元了。

栈

有 n 个数依次进栈，可以随机出栈，求出栈的所有可能情况。

输入：

3

输出：

5

当然，这是一道卡特兰数的题，但是：

(1) 万一你不记得卡特兰数的公式了呢

(2) 万一你不知道这是卡特兰数呢

所以，背公式还不如自己推导呢！

卡特兰数

简单来说，就是我们已经知道了1—i-1个数出栈的可能性想，需要我们求出i个数出栈的可能性。

假设此时第k个数已经入栈了，那么还剩下(n-k)个数，那么剩下(n-k)个数的入出栈方案数很简单就是 $h[n-k]$ ，那么已经入栈的k个数的方案数呢？ $h[k]$ ？肯定不是， $h[k]$ 表示的是前k个数出入栈种树，但是这里已经入栈的k个元素，有没有必要先出栈，再让k+1这个元素进来，所以，明显有问题。

那我们就换一种思路，假设最后一个出栈的数是k，那么说明 $[1 \cdots k-1]$ 都在k入栈之前出栈了，然后k再入栈，接下来再是 $[k+1 \cdots n]$ 入栈，然后 $[k+1 \cdots n]$ 出栈。那么此时k都把前后两部分隔离成了独立的两部分，在k入栈之前，前面部分已经入栈并且出栈了。在k之后的部分，后面部分比k后入栈并且先入栈。所以，公式就出来了：

$$h[n] = h[0] * h[n-1] + h[1] * h[n-2] + \cdots + h[n-1] * h[0]$$

卡特兰数

还有一种证明方法，也很简单：

设 $h[i][j]$ 表示栈内有 i 个，出栈的有 j 个的方案数。要达到当前这种状态，他的前一状态只有两种可能，入栈一个或者出栈一个，如果是入栈要到达 $h[i][j]$ ，那么上一状态就是 $h[i-1][j]$ ，如果是出栈到达 $h[i][j]$ ，那么就是 $h[i+1][j-1]$ 。那么递推公式就有了：

$$h[i][j] = h[i-1][j] + h[i+1][j-1].$$

注意一下，只有 j 是递增的，可能后用到后面的值，所以先枚举 j ，在枚举 i 。就可以得到正确答案。

但是前面两种时间复杂度都是 $O(n^2)$ 的，能不能更快一点，直接找公式而不是递推公式。

卡特兰数

公式也是可以推导的。画一个图就很好理解了。

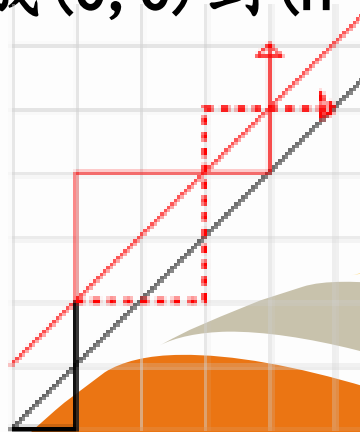
在平面直角坐标系中，x轴表示入栈次数，y轴表示出栈次数，那么卡特兰数就可以表示为从(0,0)点到达(n,n)点不越过直线 $y=x$ 的方案数。

如果不考虑限制，总方案数为 $C(2n, n)$ ，那么限制呢？我们会发现对于所有不满足条件的情况，一定是在若干次操作之出栈比入栈多一次，即这个点在 $y=x+1$ 上，我们发现如果把后面部分都相对于这条线对称，那么最后一定要到达点 $(n-1, n+1)$ 。即任何非法路径都可以变成(0,0)到 $(n-1, n+1)$ 的一条无限制的情况，那么合法方案为：

$$C(2n, n) - C(2n, n-1) :$$

$$\text{也可以转换成 } 1/(n+1) * C(2n, n)$$

$$\text{还有 } h[i+1] = (4n+2)/(n+2) * h[i]$$



卡特兰数

也可以这样考虑，对于01序列，如果不满足0的数量永远大于1的数量，那么肯定是从某一个1开始之后的数，后面有 k 个1， $k+1$ 个0。

比如011001

从第2个1开之后0的数量比1的数量多一个

那么这种情况有多少种呢？我们发现如果我们把后面的0都变成1，1都变成0，那么最终答案有 $k+1$ 个1， $k-1$ 个0. 这两种方案是一一对应的。

也就是对于一个长度为 $2*n$ 的序列，其中有 $n+1$ 个1， $n-1$ 个0，他都可以转换成唯一对应的不满足卡特兰数的序列。

比如110011，找到第一个1的数量 >0 的数量的位置，再把后面的数取反，得到的序列就是非卡特兰数的序列

那么实际上长度为 $2*n$ 的序列中，不满足条件的情况就是 $C(2n, n-1)$ 种，所以总的符合条件的数有 $C(2n, n) - C(2n, n-1)$

卡特兰数

根据前面的推导过程，我们很容易想到拉特兰数的适用范围：

(1) 一种情况的数量一直 $\geq$ 另一种情况

(2) 最终两种情况的数量一定相等

只要满足这两种条件，那么他就是卡特兰数

(1) 给出一个 n ，要求一个长度为 $2n$ 的01序列，使得序列的任意前缀中的1的个数不少于0的个数

(2) 从 $(0, 0)$ 到达 (n, n) 点，每次只能向上或者向右，问不越过 $y=x$ 的方案数

(3) 有 $2n$ 个人进入剧场，入场费5元，售货员自己没零钱有 n 个人有5元的钞票，剩下 n 个人只有10元钞票，问有多少种方法可以使得所有人都可以正常入场。

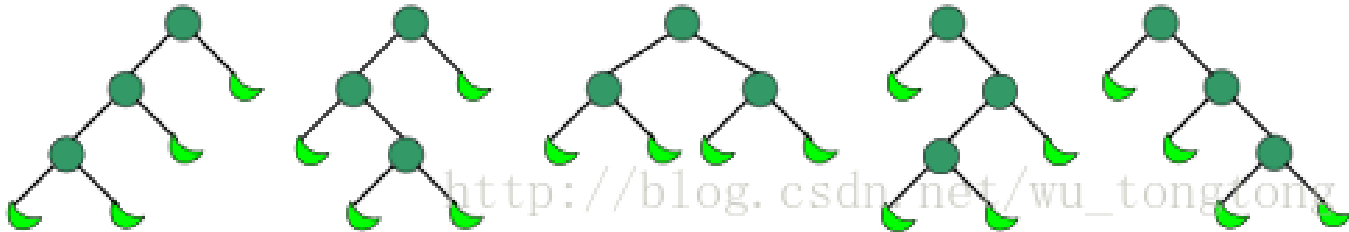
(4) n 个左括号， n 个右括号，问合法括号排列的方案数

卡特兰数

一共有 $2n+1$ 个结点，问所有不同构的含有 $(n+1)$ 个叶子结点的二叉树的个数(区分左右儿子)。

对于一个满二叉树而言，叶子结点个数=非叶子结点数+1
但是实际上，也有不是满二叉树就满足条件。

只要满足子树要么为空，要么满都可以满足条件。设叶子结点为 x ，非叶子结点为 y ；如果子树都满或者空， $x=y+1$ ，如果不满足， $x < y+1$ ；并且只有当插入到只有一个儿子结点的位置，叶子结点才会+1。所以一定是满或空这种形态。



卡特兰数

那么我们思考一下，最开始只有一个结点，就是叶子结点，然后再添加一个结点，此时 $x=y$ ，那么为了满足 $x=y+1$ ，又的让叶子结点+1。一直这样循环。当最后满足有 n 个叶子结点， n 个非叶子结点的时候，最后再添加的那一个结点是固定的，所以实际上还是卡特兰数。只是：

$h[i]$ 表示有 $n+1$ 个叶子结点的二叉树的个数。

卡特兰数

把一个凸 n 边形划分成 $(n-2)$ 个三角形的方案数：

卡特兰数，先固定一条边，再从剩下点里面选择一个点，那么这样构成的一个三角形会把整个凸 n 边形分成两个独立的多边形，那么这里两个多边形的划分方案乘起来就是当前这种情况的总方案，那么由于选择的点只要不和这条边的两个点重合，那么都可以。实际上就是 $h[i+2]$ 表示卡特兰数的第 n 项的答案。



无平方因子数

给出正整数 n 和 m ，区间 $[n, m]$ 内的“无平方因子”的数有多少个？整数 p 无平方因子当且仅当不存在 $k > 1$ ，使得 p 是 k^2 的倍数。 $1 \leq n \leq m \leq 1e12, m - n \leq 1e7$ 。



无平方因子数

其实该题和前面筛素数的做法是一样的，首先，如果一个数是平方因子数，那么他肯定有一个因子 $\leq \sqrt{n}$ ，所以我们还是只需要枚举 $\sqrt{n}$ 以内的素数就可以了。

同样，我们只需要枚举素数的平方，在枚举倍数，因为非素数的平方都可以有素数的平方*倍数得到。

同样，该题区间所处的位置很大，但是区间长度很小，所以还是要注意*倍数的时候，直接保证在区间内，所以该题实际上就是一道筛选素数的裸题。



因子数

求出 $[1, n]$ 之间每个数的因子的集合。
 $n < 10^6$



因子数

如果我们还是按照前面的做法分别求出每个数的因子有哪些，那么时间复杂度就是 $n * \sqrt{n}$ ，所以我们需要找到一个更快的办法。

既然是区间统计，那么我们就没有必要一个一个算，一个数 a 的倍数是 b ，那么 a 肯定就是 b 的因子，所以我们可以把求用试除法求因子数转换成用倍数来求解。

因子数

```
vector<int> v[1000100]; // 可变数组
int n;
int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=n/i;j++)
            v[i*j].push_back(i);
    }
    for(int i=1;i<=n;i++)
    {
        for(int j=0;j<v[i].size();j++)
        {
            printf("%d ",v[i][j]);
        }
        printf("\n");
    }
}
```

poj 3090

在一个平面直角坐标系的以 $(0,0)$ 为左下角， (N,N) 为右上角的矩形中，除了 $(0,0)$ 之外，每个坐标上都插着一个钉子，求在 origin 向四周看去，能够看到多少个钉子，一个钉子能被看到，当且仅当连接它和 origin 的线段上没有其他钉子。 $N \leq 1000$.

poj 3090

自习观察就会发现，除了 $(0, 0)$, $(0, 1)$, $(1, 0)$ 之外，其他点 (x, y) 被看到当且仅当 $\gcd(x, y) = 1$ ，即横纵坐标互质，也及时对于一个数 x ，所有与 x 互质的数有多少个，所以很明显就是欧拉函数。

所以该题就是欧拉函数的线性筛之和+3就是答案



因子数

1— n 之间与 n 互质的数之和是多少？



欧拉函数

我们仔细观察 n 以内的质数有哪些，以及之和是多少就会发现一个规律，即 $f(n) * n / 2$.

怎么证明呢？很简单

如果一个数 a 与 n 互质，那么 $(n-a)$ 也与 n 互质

很容易证明， $(n-a) = k * x$ $n = k * y$

那么 $a = k(y-x)$ ，如果 $(n-a)$ 与 n 不互质，那么 a 与 n 也不互质，所以证明不成立。

反素数

题目描述

[展开](#)

对于任何正整数 x ，其约数的个数记作 $g(x)$ 。例如 $g(1)=1$ 、 $g(6)=4$ 。

如果某个正整数 x 满足： $g(x) > g(i)$ $0 < i < x$ ，则称 x 为反质数。例如，整数1，2，4，6等都是反质数。

现在给定一个数 N ，你能求出不超过 N 的最大的反质数么？

输入格式

一个数 N ($1 \leq N \leq 2,000,000,000$)。

输出格式

不超过 N 的最大的反质数。

输入输出样例

输入 #1

[复制](#)

1000

输出 #1

[复制](#)

840

反素数

这题看上去是一道数论题，实际上确定一道带技巧性的搜索题，必须要掌握里面的规律，否则爆搜肯定T

结论一：1—N中的反素数就是1—N中因子最多的数中最小的那一个

结论二：根据数据范围，因子数不会超过10个，因为 $2*3*5\cdots 31 > 2e9$

结论三：相同质因子最多30个，因为 $2^{31} > 2e9$

结论四(重要结论)：反素数的质因子一定是连续若干个最小的质数，并且指数单调递减。设连续质数分别为 $x_1, x_2, x_3\cdots x_n$ ，不存在 $x_i \mid x_{i+2}$ 这种情况

反素数

证明：

如果存在反素数 $x_1 * x_2 * \dots * x_i * x_{i+2}$ ，那么必有 $x_1 * x_2 * \dots * x_i * x_{i+1}$ 值比前面一个数小并且因子数一样。

同样，如果连续质因子的质数不递减，也可以用前面的替换方法来得到一个比当前数小但是因子数量一样多的数。

所以我们直接跑深搜，只需要筛选出前10个质数，搜索的时候指数递减搜索就可以了。

容斥原理

给出 m 个数 $a[1], a[2] \cdots a[m]$, 求 $1 \cdots n$ 中有多少个数不是 $a[1], a[2] \cdots a[m]$ 的倍数。

输入:

10 2

2 3

输出:

3

$N \leq 10^9, m \leq 20, a[i] \leq 10^9$

MSOJ 1332 : 计数

容斥原理

这其实就是一道容斥原理的模板题，我们需要求多少个数不是给定的这些数的倍数，那么有多少个数是这些数的倍数呢？如果直接枚举判断肯定超时，那么我们可以每次用给出的数的倍数去筛掉这些数 ($a[i], 2*a[i] \cdots k*a[i]$)，但是最大的问题在于重复，比如一个数很有可能是多个数的倍数，怎么才能既不遗漏又不重复筛选呢？

实际上这就是容斥原理的板子题了，先用 $a[1]$ 去筛选出 $n/a[1]$ 个数，再用 $a[2]$ 去筛选，可以筛选出 $n/a[2]$ 个数，那么重复的怎么算呢？ $n/(a[1]*a[2])$ ，并不是，而是 $\text{lcm}(a[1], a[2])$ ，那么当用第3个数筛选的时候呢？不要忘记还要筛选出 $\text{lcm}(a[2], a[3])$ 和 $\text{lcm}(a[1], a[3])$ ，怎么实现呢？很简单用dfs枚举+剪枝。

容斥原理

```
long long getf(long long x,long long y)
{
    return x*y/gcd(x,y);
}
void dfs(int step,int k,long long lcm)
{
    if(lcm>n)//最小公倍数大于n, 不在考虑了
        return ;
    if(step==m+1)
        //最多选到了m个数, 但是其中可能只选了k个数, 我们实际上是求k个数的交集
        {
            if(k&1)//补集就是奇减偶加
                ans-=n/lcm;
            else
                ans+=n/lcm;
            return ;
        }
    dfs(step+1,k,lcm);//当前第step个数不选, 那此时我选择的数的个数仍然是k
    dfs(step+1,k+1,getf(lcm,a[step]));
    //选择当前第step个数, 那么选择了k+1个数, gcd为前面的lcm和该数的lcm
}
```

魔数

魔数(number.cpp)

【题目描述】

小林和亮亮是好朋友。小林有一个幸运数字 a ，亮亮有一个幸运数字 b 。一个数字称之为“幸运数”当且仅当它只由 a 和 b 组成。小林称一个数为“魔数”，当且仅当这个数各数位之和为“幸运数”，且这个数字本身也是“幸运数”。

举个例子：小林的幸运数字为 2，亮亮的幸运数字为 6，那么 23 不是“幸运数”，而 26、222 是“幸运数”。进一步，222 是“魔数”（因为 $2+2+2=6$ ），而 26 不是“魔数”（因为 $2+6=8$ ）。

亮亮想要知道，有多少个 n 位的“魔数”（一个 n 位数不包含前导 0），由于这个数字会很大，所以亮亮只关心这个数模 1000000007 （ 10^9+7 ，是一个质数）的结果。



魔数

排列组合+快速求逆元



candy

1. Candy ↵

(candy. cpp/c/pas) ↵

【问题描述】 ↵

糖糖有一个整数A,设A的十进制表示为 $a_1a_2a_3\dots a_n$; ↵

定义 $Rotate(a_1a_2a_3\dots a_n) = a_2a_3\dots a_na_1$ ↵

糖糖利用这个方法生成了n个数字串,其中: ↵

$b_1 = A, b_2 = Rotate(b_1), \dots, b_n = Rotate(b_{n-1})$ ↵

糖糖对于这些数字串在十进制下的和很感兴趣,现设 $\sum b_i = S$ ↵

她想知道S的最小非1的因子是多少 ↵



练习题

洛谷	P3197	越狱
洛谷	P2290	树的计数
洛谷	P2822	组合数问题
洛谷	P3048	牛的IDCow IDs
洛谷	P1072	hankson的趣味题





练习题

NOIP 2016 组合数问题

NOIP 2011 计算系数

洛谷 2523 problem C





练习题

POJ1061:青蛙的约会

NOI2002 荒岛野人

POJ 2142 天平

洛谷 1170 兔八哥和猎人





练习题

洛谷	1044	栈
洛谷	1976	鸡蛋饼
洛谷	2532	树屋阶梯
洛谷	3200	有趣的数列
洛谷	4981	父子

